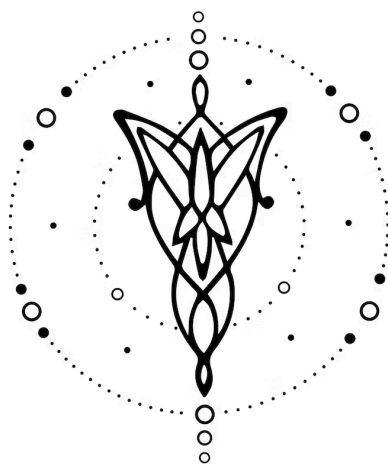




Entrega Final

Trabajo Práctico Especial

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

Nombre	Apellido	Legajo	E-mail
Augusto	Cicciomessere	62267	acicciomessere@itba.edu.ar
Josefina	Miguez	63132	jmiguez@itba.edu.ar

1. Introducción	1
2. Modelo Computacional	1
2.1. Dominio	1
2.2. Lenguaje	2
3. Implementación	3
3.1. Frontend	3
Análisis Léxico con Flex	3
Análisis Sintáctico con Bison	4
3.2. Backend	4
3.3. Dificultades Encontradas	5
4. Futuras Extensiones	5
5. Conclusión	6

La solución y su documentación serán versionadas en: [DocuJSON](#)

1. Introducción

En este informe detallamos el desarrollo del Trabajo Práctico Especial de la materia. El mismo consiste en el diseño e implementación de un compilador capaz de generar documentación en formato HTML a partir de un archivo con estructura similar a JSON. El propósito central es facilitar la creación y personalización de documentación técnica, mostrando de forma clara los métodos, variables, parámetros y relaciones del código fuente, junto con descripciones y estilos visuales configurables.

Para la implementación del trabajo se utilizó el lenguaje de programación C y con las herramientas Flex y Bison para el análisis léxico y sintáctico, respectivamente.

2. Modelo Computacional

2.1. Dominio

El lenguaje es similar a JSON con etiquetas reservadas que se utilizan para crear y personalizar documentación de forma tal que se pueden agregar tanto métodos como variables y parámetros para los distintos métodos, donde se aclaren los componentes del código y teniendo dentro de cada uno la posibilidad de darle una descripción y un estilo base a la documentación en sí.

En el caso que salga correctamente el código, el compilador genera como salida un archivo HTML que incluye tanto el contenido como su correspondiente formato visual, respetando las especificaciones definidas en el archivo fuente. El archivo generado es navegable, es decir los métodos relacionados son links a otras funciones dentro de la documentación. De este modo, se obtiene una

documentación completa, clara y con un diseño coherente, generada de manera automática a partir de una fuente legible y flexible.

2.2. Lenguaje

El lenguaje diseñado permite estructurar y personalizar documentación técnica a través de un conjunto de etiquetas predefinidas. Las principales funcionalidades y construcciones disponibles son:

- A partir de la etiqueta **method**, es posible documentar las funciones del código, detallando obligatoriamente su descripción y su tipo. Adicionalmente, se puede detallar los parámetros que reciben dichas funciones, especificando descripción y tipo, las variables internas que posee (también detallando descripción y tipo), y los métodos con los que se relaciona. Pueden existir funciones bajo el mismo nombre pero solamente si estas tienen distintos parámetros.
- A partir de la etiqueta **variables**, se puede documentar las variables globales del programa teniendo estas descripción y tipo, siendo estas variables únicas a partir de su nombre ya que no pueden definirse 2 variables con el mismo nombre.
- Se podrá agregar estilo al texto para el nombre de las funciones y otro a la descripción de la misma. De igual manera se puede hacer con las variables globales. El estilo debe ser el propio de HTML, respetando las palabras válidas de CSS y escritas en el siguiente formato:

...ejemplo:

```
style: {
    methods: {
        title: "etiqueta": "valor"; "etiqueta": "valor"; "etiqueta": "valor";,
        description: "etiqueta": "valor"; "etiqueta": "valor";
    },
}
```

- Las etiquetas del lenguaje son **methods**, **variables**, **style**, **params**, **description**, **regex**, **range**, **related** y **type**.

- Para las etiqueta type se aceptan los siguientes tipos: `string`, `int`, `float`, `boolean`, `object`, `array`, `number`, `void`, `Date`, `function`
- No se incluyen operadores lógicos, aritméticos y relacionales, ya que no forman parte del objetivo del lenguaje.
- Se aceptan archivos sin especificación de estilo.
- Se aceptan archivos que solo incluya variables o solo incluya funciones.
- No se aceptan archivos vacíos o solo con estilo.

3. Implementación

3.1. Frontend

El frontend fue implementado utilizando las herramientas Flex y Bison.

Análisis Léxico con Flex

En el archivo `FlexPatterns.l` identificamos los componentes léxicos que aparecen en el archivo de entrada. Por un lado están las palabras y símbolos clave como “`methods`”, “`params`”, o bien “`;`”, “`,`”, etc. Por el otro, también se especifican regiones como `STRING_REGION` que es cuando se pasa una cadena de caracteres entre comillas, o `MULTILINE_COMMENT`, el cual abarca los comentarios que están encerrado entre “`/*`” y “`*/`”, los cuales no se tienen en cuenta en el análisis de la estructura del JSON.

Las acciones de dichos componentes se detallan en el archivo de `FlexAction.c` el cual enlaza cada lexema con su correspondiente token semántico que será procesado por el analizador sintáctico Bison, detallado a continuación.

Análisis Sintáctico con Bison

El archivo `BisionGrammar.y` contiene la gramática del lenguaje de entrada. En este archivo especificamos algunas reglas de lo que acepta nuestro lenguaje o no: como por ejemplo, que el archivo de entrada tipo JSON sea válido aunque no tenga una sección de estilo, o funciones que no tienen variables internas, o tenga más de una.

A su vez, construimos la estructura de un Árbol de Sintaxis Abstracta (AST) que representara de manera estructurada y jerárquica la información del archivo fuente.

Cada regla de producción del archivo de Bison incluye una acción semántica que definimos en `BisionActions.c` que crea nodos del AST.

Una vez con el analizador léxico y sintáctico implementados, escribimos tests de aceptación y de rechazo para comprobar que nuestro programa funcionara correctamente, aunque todavía no tuviera la implementación de validación ni de generación de código.

3.2. Backend

Para la sección de domain-specific se realizó un validador el cual de forma recursiva analiza distintos casos posibles previo a la generación de código para los métodos, las variables y el estilo. Entre estos se encuentran chequeos de métodos repetidos (con el mismo nombre y los mismos parámetros), chequeos de variables repetidas (con el mismo nombre), verificar que las funciones relacionadas existan, se valida el contenido del regex, rango y los styles, al igual que los valores ingresados en type pues aceptamos tipos específicos. Para ayudar a las validaciones del estilo se realizaron funciones auxiliares para validar tanto la etiqueta como el valor dentro de `utils.c`.

Finalmente se verifica el contenido de la descripción para evitar texto en blanco al igual que la longitud de estas.

En la etapa de generación de código se modificó el archivo `Generator.c` para que transforme el archivo fuente en un HTML. Para eso, separamos el HTML en tres secciones: la sección prólogo, el cual tiene el principio típico de un HTML y algunas consideraciones de estilo que tomamos, la sección del programa, que corresponde más a la información del archivo, y finalmente el epílogo para cerrar algunas etiquetas.

Para la sección del programa, se traduce cada nodo del AST formado en el Frontend a su representación visual en HTML, el cual se imprime en consola al hacer `/script/ubuntu/start.sh <program>`

3.3. Dificultades Encontradas

La parte que más complicaciones nos trajo a la hora de la implementación, fue la de transformar la parte que se especifica el estilo en el archivo tipo JSON al estilo real del HTML. Nos encontramos con que estábamos parseando de manera incorrecta el estilo, considerando válido simplemente una cadena de caracteres, pero luego notamos que debía tener una estructura específica para que se lograra ver de manera correcta en el explorador. Para eso tuvimos que cambiar la forma en el Bison para que aceptara solo el formato correspondiente el cual es “etiqueta”:”valor”;;, donde etiqueta y valor son palabras aceptadas por CSS.

4. Futuras Extensiones

Posibles futuras extensiones podrían ser que se agrande la capacidad de personalización de estilo, que sería permitir más cosas en la sección de estilo en el JSON y no solo al título y la descripción de los métodos y variables.

Sería óptimo en una documentación técnica poder manejar más información que ayude a la comprensión de las funciones implementadas, como ejemplos o gráficos.

Además sería interesante explorar la implementación de aceptación de estructuras de datos más complejas, como arrays.

Finalmente creemos que una gran adición sería el uso de librerías para poder llevar la personalización un paso más y dar la opción de agregar botones a urls propias del creador de la librería, que los métodos sean accordions y que muestren su información al clickear, entre otros.

5. Conclusión

En resumen, este trabajo práctico permitió abordar el diseño e implementación de un compilador capaz de generar documentación técnica en formato HTML a partir de una entrada estructurada similar a JSON. Durante el desarrollo, se utilizaron herramientas como Flex y Bison, lo cual facilitó la aplicación y profundización de los conceptos de análisis léxico y sintáctico estudiados en la materia. El resultado final cumple con los objetivos propuestos, brindando una solución funcional y adaptable para la automatización de documentación técnica.